

CAPTURE WHAT MATTERS.

One need. One Windows Forms utility. One tested GitHub handoff.

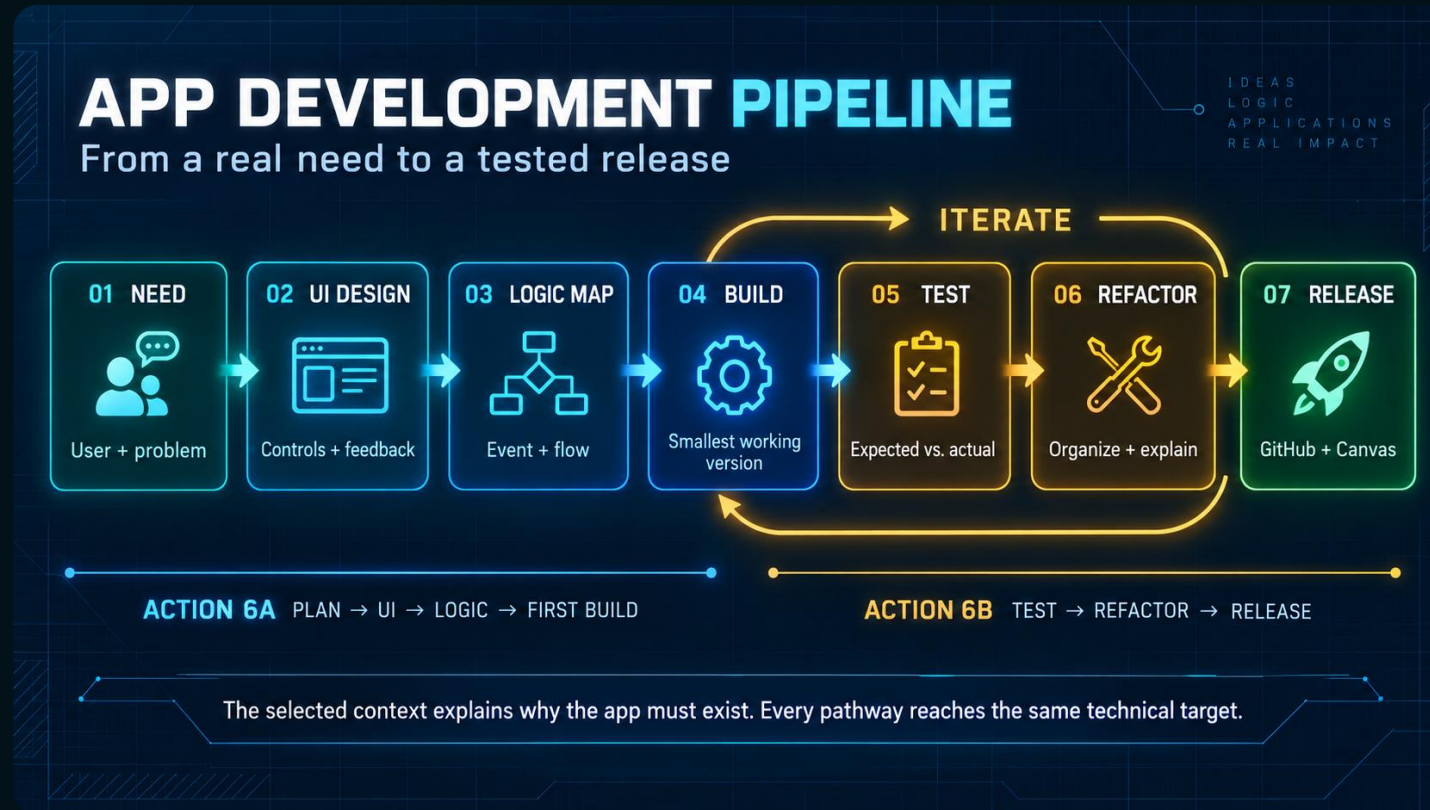
6A Need + interface

6B Capture + test

The story explains why. The engineering proves how.

A need becomes a tested utility.

Build, Test, and Refactor form an iteration loop—not a one-way finish line.



ACTION 6A

ACTION 6B

WEEK 6

Three reasons. One technical target.

Your choice changes the purpose, labels, documentation, and ethical boundary.



VOID RUNNERS

Preserve an unstable Ashwake terminal record before it resets or changes.



ORIGINAL STORY

Protect visual information that matters to someone in your own world.



TECHNICAL ROUTE

Document software behavior, a test result, or a support problem.

Shared target: C# Windows Forms + primary-screen capture + PNG save + visible status + tested GitHub repository

Run the empty form before adding capture code.

This separates environment trouble from code trouble.

01 Create

Choose C# Windows Forms App—not WPF—and use the framework approved in class.

02 Run

Launch the untouched form. The blank window proves the template and workload work.

03 Checkpoint

Create or connect Git. Commit and push the empty working form before editing.

Project

ScreenCaptureUtility

Form

MainForm

Controls

btnCapture
btnSave
picPreview
lblStatus

Recovery

Missing template?
Check .NET desktop
development workload.

The user should know what to do and what happened.

Use the Visual hat before the Logical hat.

1 CAPTURE SCREEN

2 PREVIEW IMAGE

3 SAVE PNG

4 READ STATUS

[Capture screen] [Save PNG] Ready. Choose Capture screen.

PICTURE PREVIEW

Names explain roles: btnCapture · btnSave · picPreview · lblStatus

The button asks. A method does the work.

The form handles interaction; the service handles capture.

- 1 Button click
- 2 Click handler
- 3 Capture method
- 4 Bitmap
- 5 Preview
- 6 Save choice
- 7 Status

MainForm

```
btnCapture_Click(...)
{
    currentCapture = captureService
        .CapturePrimaryScreen();
    picPreview.Image = currentCapture;
}
```

ScreenCaptureService

```
CapturePrimaryScreen()
{
    read screen bounds
    create Bitmap + Graphics
    CopyFromScreen(...)
    return image
}
```

Move pixels into a bitmap—and release temporary resources.

Explain each object before copying code.

A Primary screen bounds

Provides the primary display location and size.

B Bitmap

Stores the captured pixels in memory.

C Graphics

Creates a temporary drawing surface connected to the bitmap.

D CopyFromScreen

Copies visible pixels from the screen into the bitmap.

```
using Graphics graphics = Graphics.FromImage(image);
graphics.CopyFromScreen(bounds.Left, bounds.Top, 0, 0, bounds.Size);
return image;
```

WHY USING? Graphics is temporary. Dispose it when the capture method ends.

Success, cancellation, and failure are different routes.

SaveFileDialog gives the user control of location and file name.

SUCCESS

Save PNG; show the file name.

CANCEL

Keep the capture; say that save was cancelled.

FAILURE

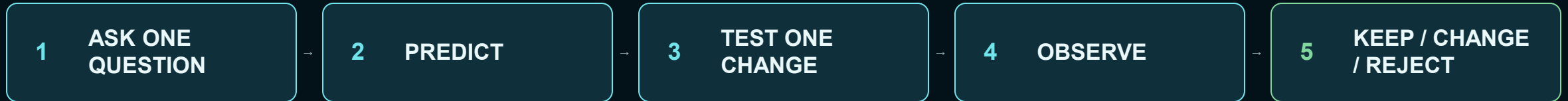
Show the exact problem; keep the app usable.

```
Filter = "PNG image (*.png)|*.png"  DefaultExt = "png"  currentCapture.Save(dialog.FileName, ImageFormat.Png);
```

PRIVACY GATE Never submit passwords, tokens, private messages, grades, or unrelated personal information.

AI advice becomes evidence only after you verify it.

Keep the request narrow and preserve the working core.



BEGINNER AI PROMPT

Explain ONE selected method line by line. Identify variables, objects, inputs, output, and disposable resources. Suggest ONE small error-feedback improvement and ONE test. Preserve my working core. Do not replace the project.

TEST ROUTES

successful save · cancelled save · blocked destination · second capture

6A and 6B advance the same weekly build.

Reuse class evidence. Do not create duplicate reports.

A Need + interface

Context map, reading start, ethics boundary, prototype, first event, peer check, commit and push.

B Capture + test

Capture and PNG save, AI record, test routes, refactor, README peer run, commit and push.

W Final handoff

Brief, workbook page 117, reused class evidence, README, screenshots, receipt, final push.

1 COMMIT 2 PUSH 3 OPEN GITHUB 4 VERIFY FILES + ACCESS 5 SUBMIT URL IN CANVAS

Both Action 6A and Action 6B must be accepted before the weekly assignment is accepted for grading, subject to the syllabus and approved accommodations.